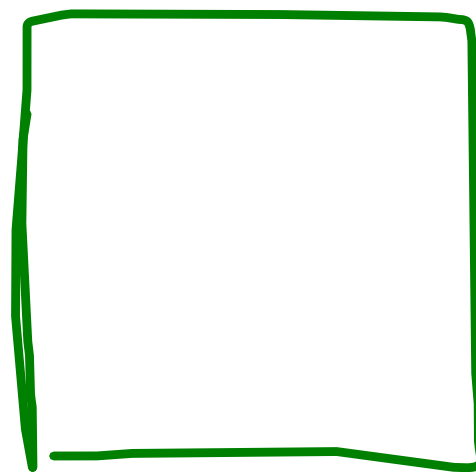
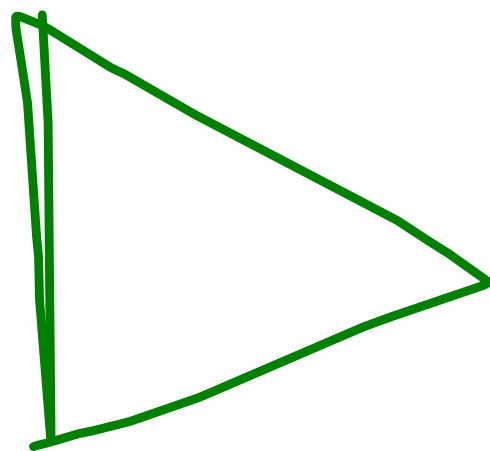
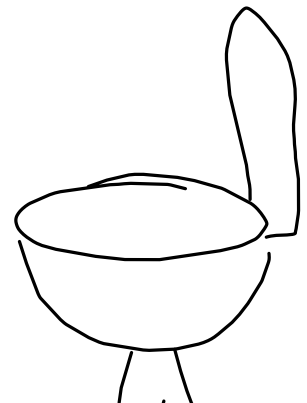
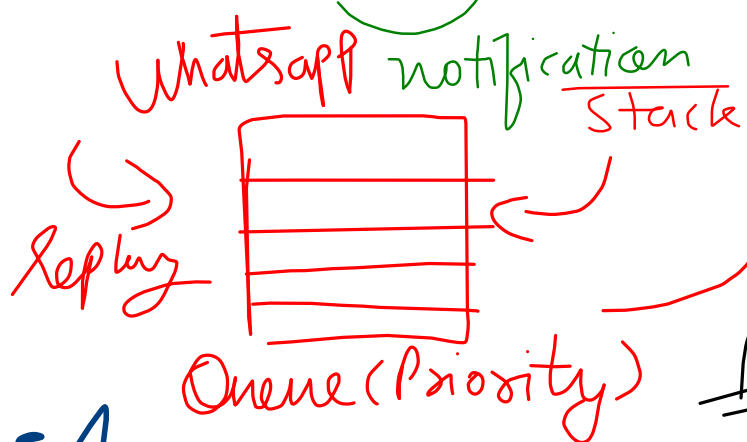
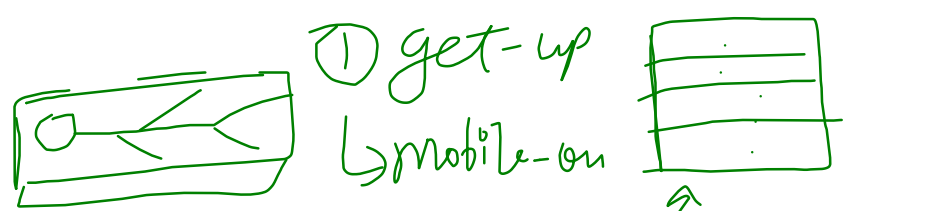
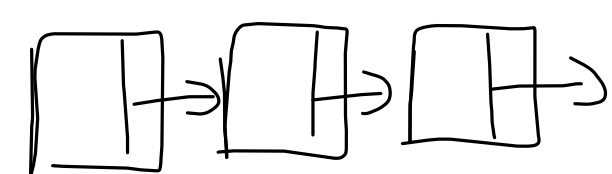


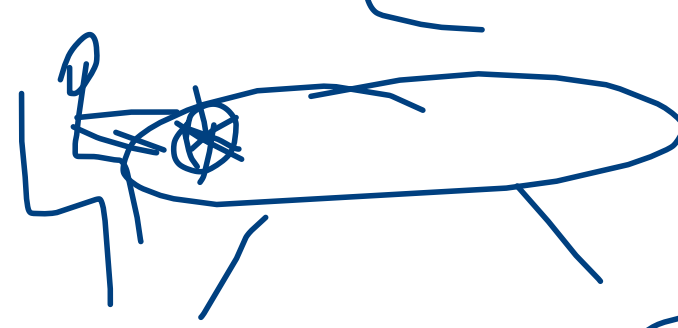


Reels / Shorts

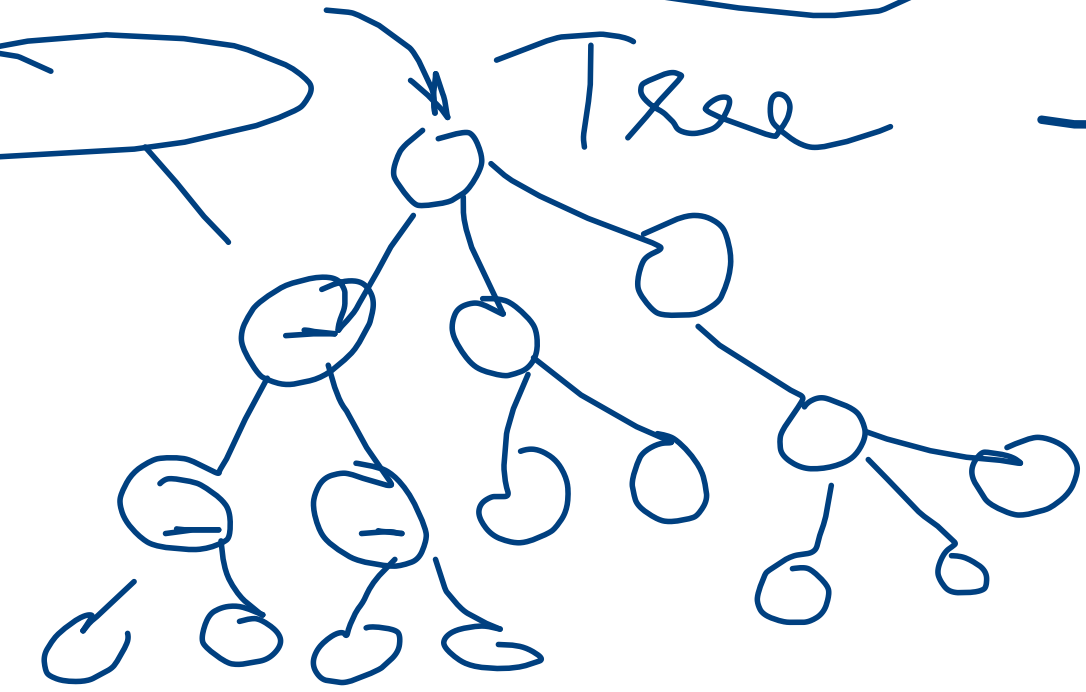


Linked List

Social



Tree



home Graph

CONC



Stack:

push(e): accept element on top of stack.

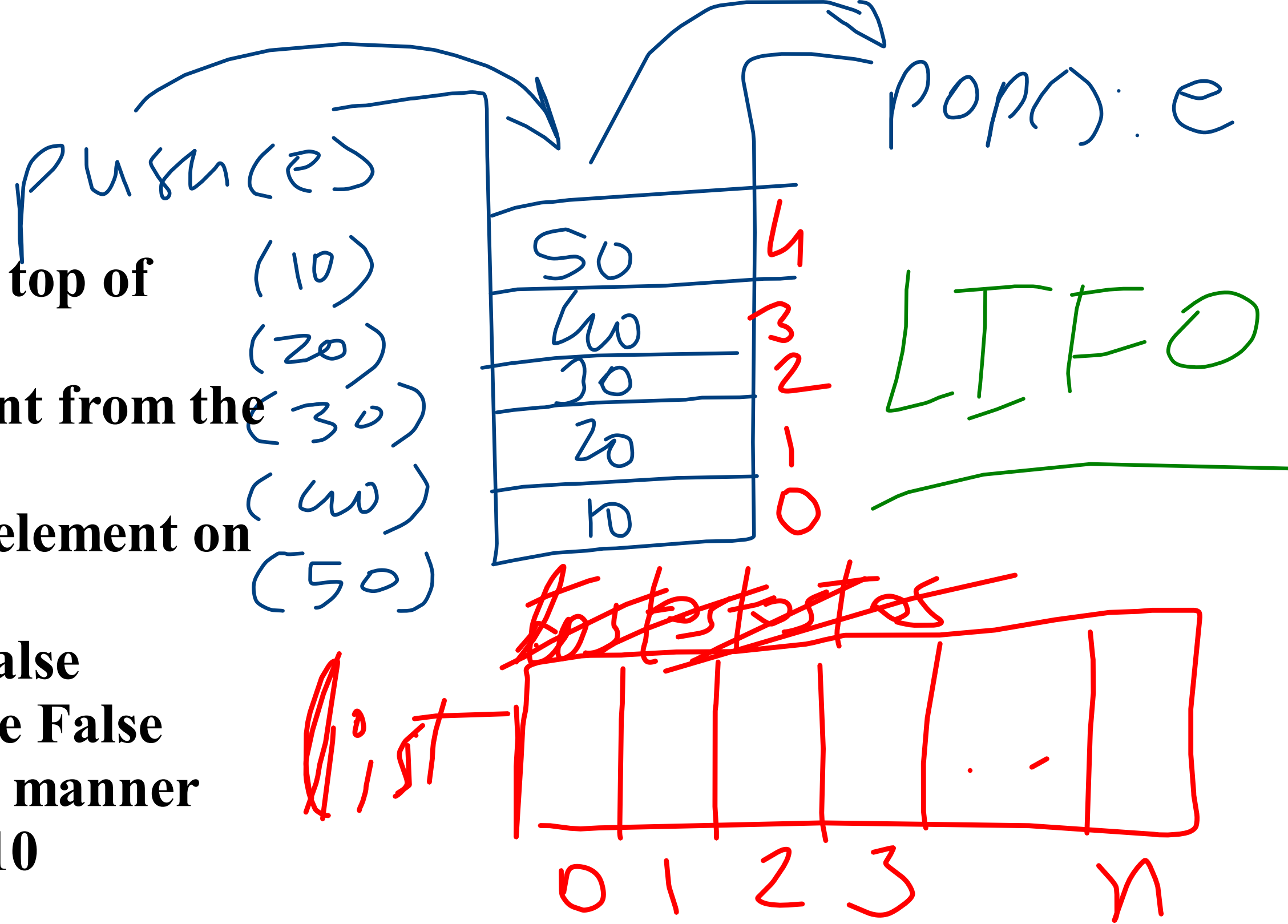
pop(): Removes an element from the top of the stack.

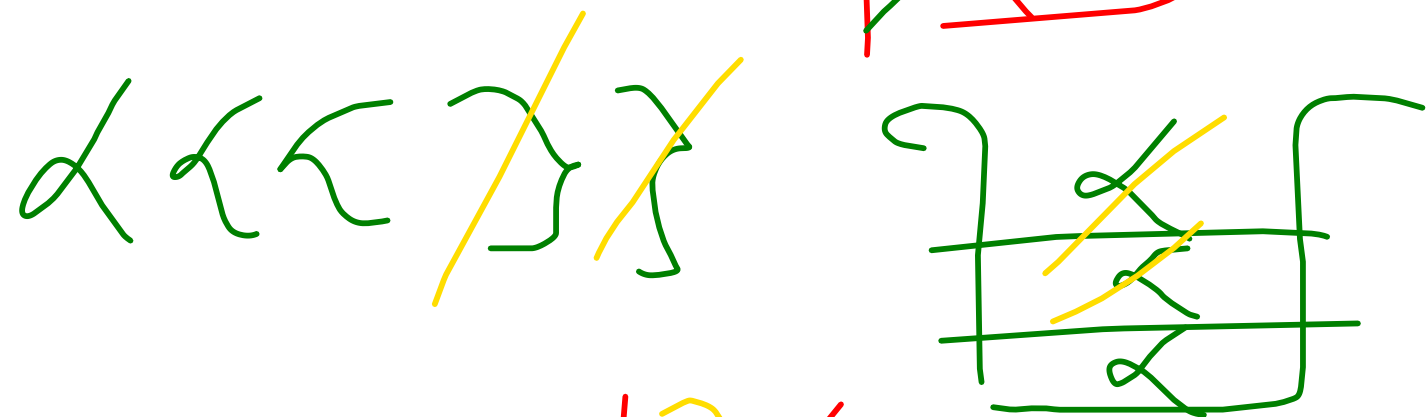
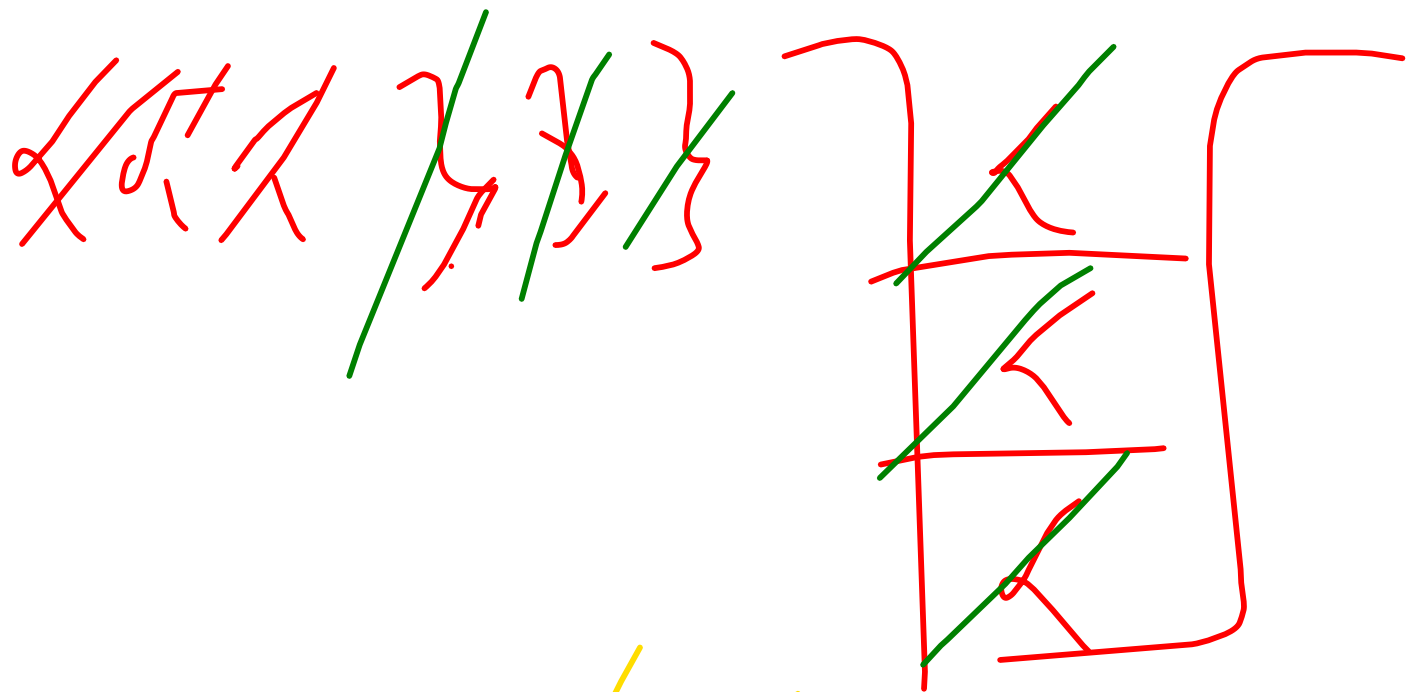
peek(): Only returns the element on top of the stack.

Is_full(): True if full else False

Is_empty(): True if empty else False

print_stack(): prints in lifo manner
50,40,30,20,10





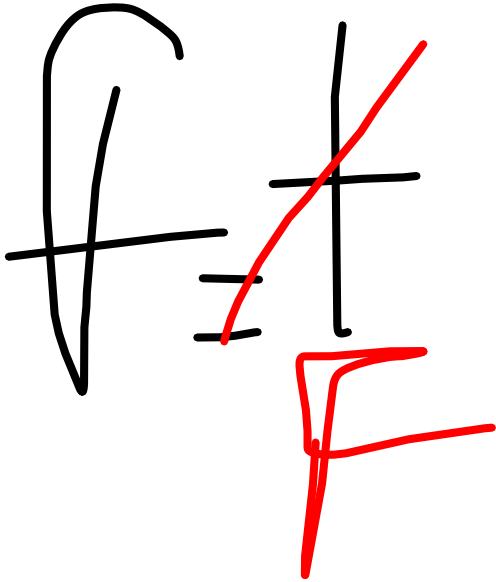
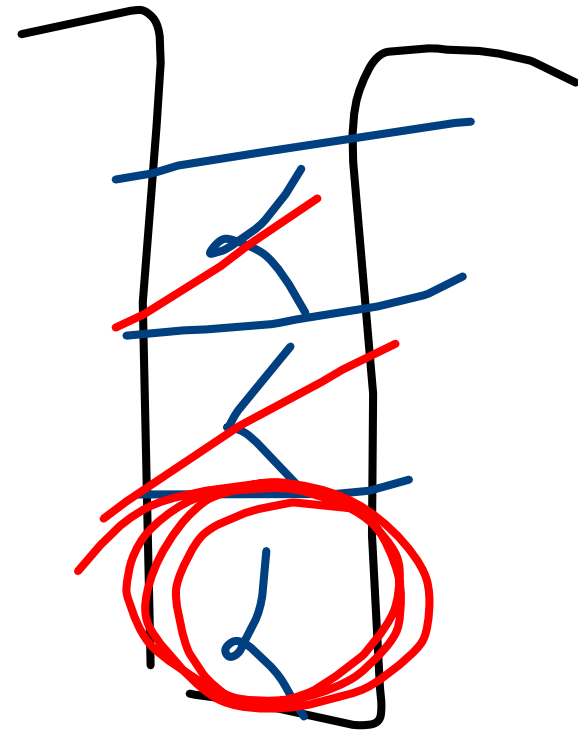
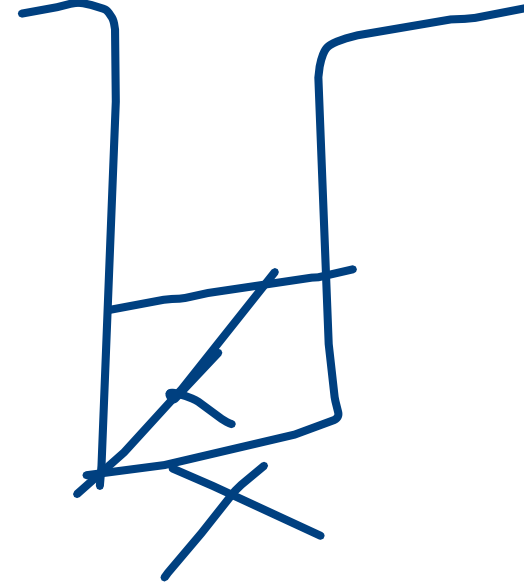
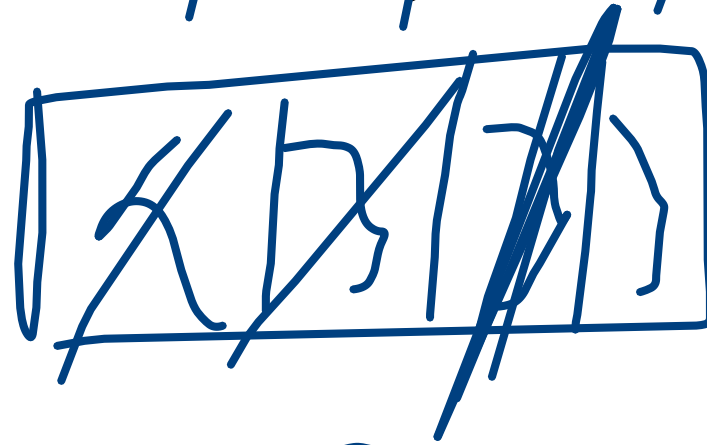
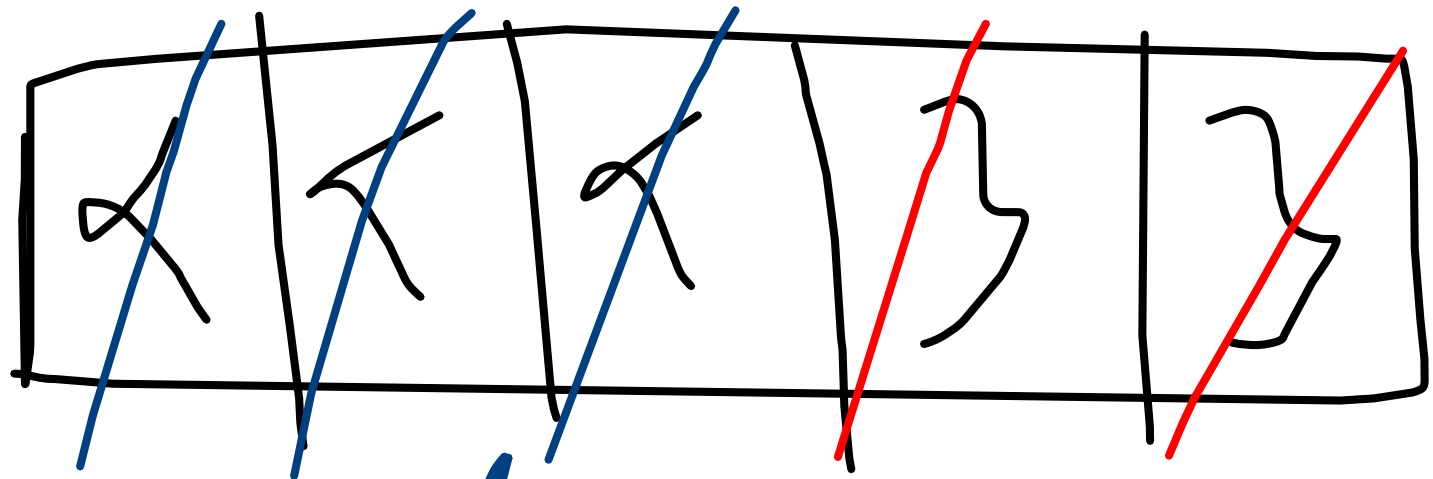
← empty!

```

line="{{{} }}"
s=DS_Example()
flag=True
for char in line:
    if char=='{':
        s.push(char)
    elif char=='}':
        if not s.is_empty():
            s.pop()
        else:
            print("Error:")
            flag=False
            break
if s.is_empty()!=True:#stack not empty
    flag=False

if flag:
    print(f"{line} is Correct")
else:
    print(f"{line} is not Correct")

```



5

$$10 / 2 = 0$$

$$10 / 2 = 5$$

$$\begin{array}{r} 2 \overline{) 10} \\ - 10 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 2 \overline{) 5} \\ - 4 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 2 \overline{) 2} \\ - 2 \\ \hline 0 \end{array}$$

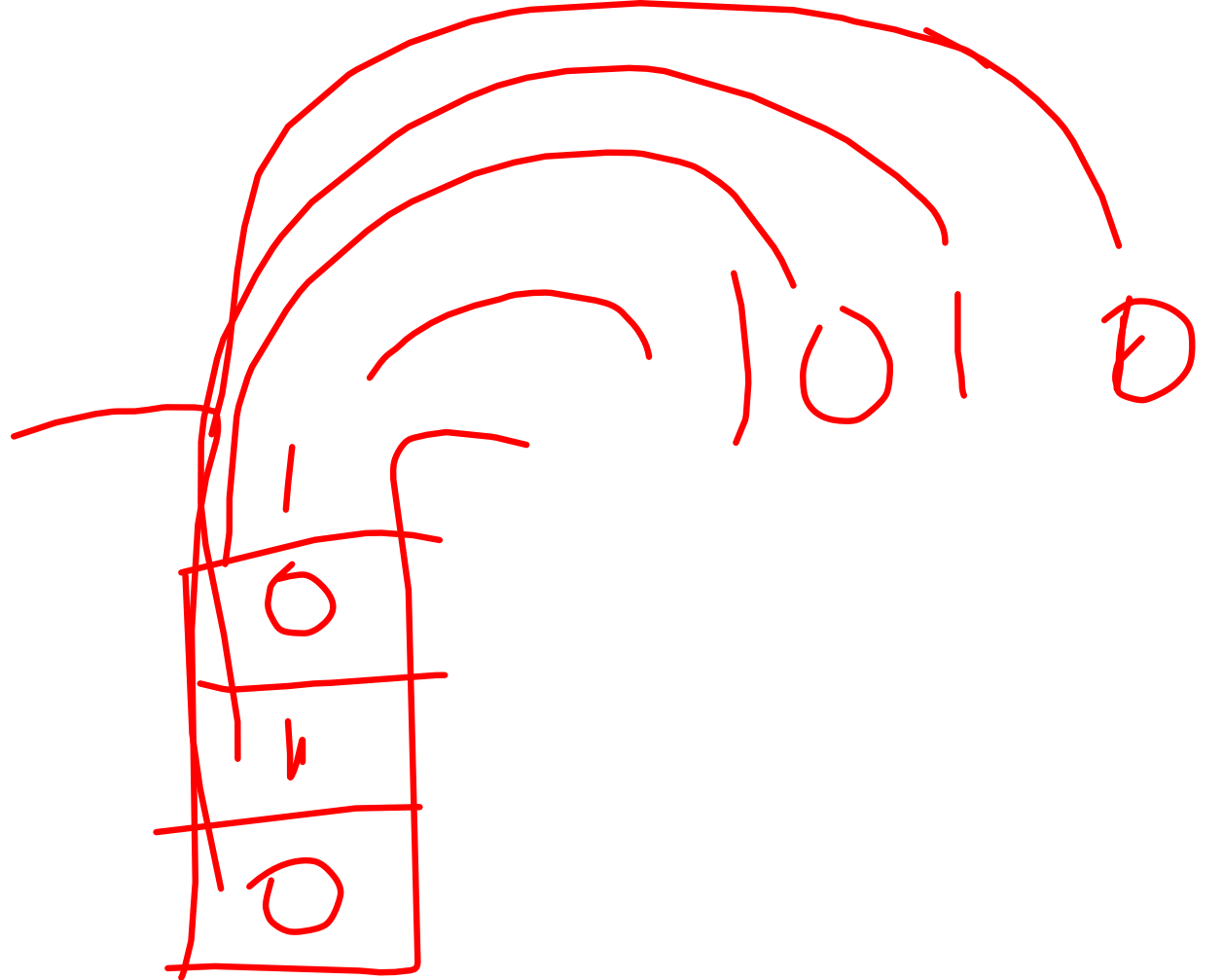
$$\begin{array}{r} 2 \overline{) 1} \\ - 0 \\ \hline 1 \end{array}$$

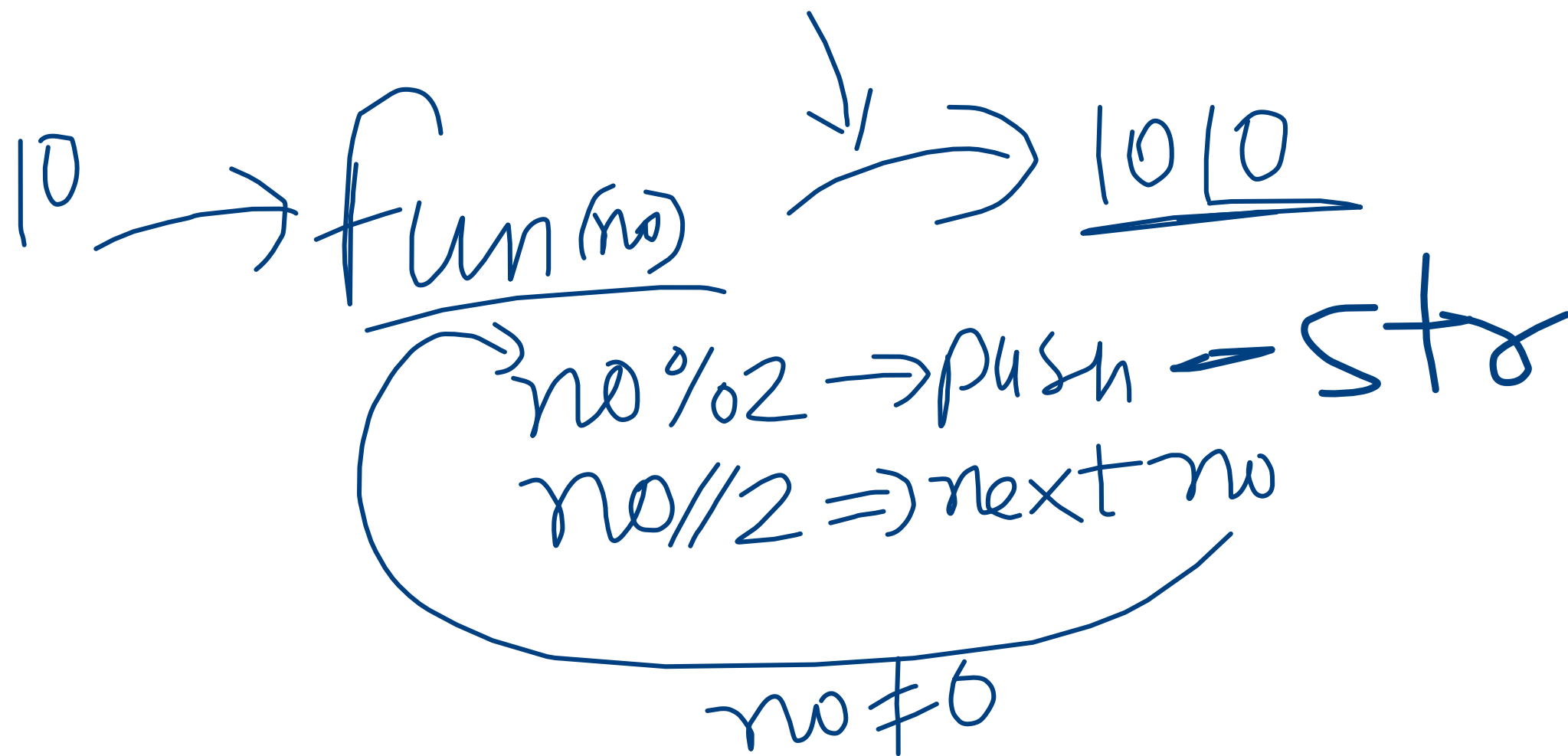
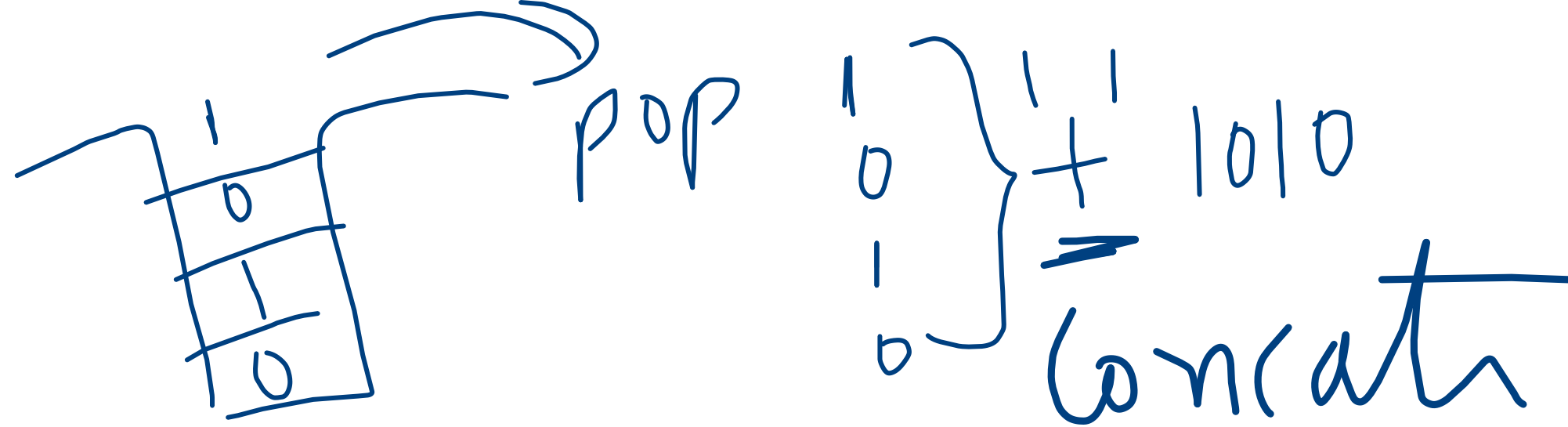
$$\begin{array}{r} 2 \overline{) 0} \\ - 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 2 \overline{) 0} \\ - 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 2 \overline{) 0} \\ - 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 2 \overline{) 0} \\ - 0 \\ \hline 0 \end{array}$$






```

1 #Convert decimal to binary:
2 #input:decimal number o/p:binary string
3 #convert_bin(10):--->1010
4 def convert_bin(no):
5     bin=[]
6     while no>0:
7         digit=str(no%2)
8         bin.append(digit)
9         no=no//2
10    return "".join(reversed(bin))
11

```

bin

0	1	0	1
---	---	---	---

$10/2=5$ | $5/2=2$ | $2/2=1$ | $1/2=0$
 $10\%2=0$ | $5\%2=1$ | $2\%2=0$ | $1\%2=1$
 $5/2=2$ | $2/2=1$ | $1/2=0$
 $5\%2=1$ | $2\%2=0$ | $1\%2=1$
 $10\%2=0$ | $5\%2=1$ | $2\%2=0$ | $1\%2=1$
 0101 $\rightarrow$ 1010 $\rightarrow$ 10+1+0 = 1010

```

1 convert_bin(13)

```

$\text{fact}(1)$

$$1 \times \text{fact}(3) = 24$$

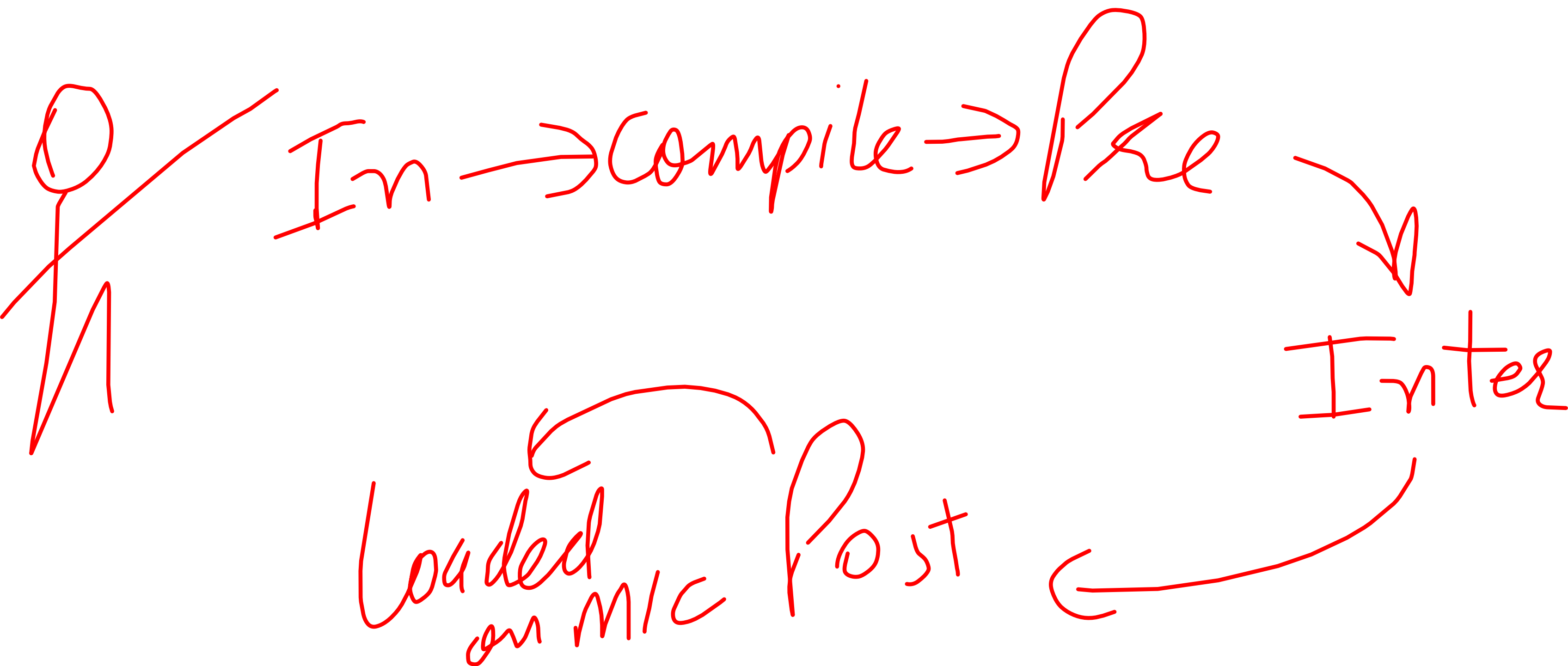
$\downarrow$

$$3 * \text{fact}(2) = 6$$

$\downarrow$

$$2 * \text{fact}(1) = 2$$

$\cup$



$$a + \underline{b * c} - d$$

Pre

$$a + \underline{*bc} - d$$

$$a + \beta = +a\beta$$

$$+a * bc - d$$

$$A - d = -Ad$$

$$- +a * bcd$$

$$a + \underline{bc*} - d$$

$$\underline{abc*} + -d$$

$$\underline{abc*} + d -$$

$$(a + b) * (c - d) / e + f$$

$$\underline{+ab * -cd} / e + f$$

$$\underline{* +ab - cd} / e + f$$

$$A / e = / Ae$$

$$\underline{/ +ab - cde} + f$$

$$B + f = +Bf$$

$$\underline{+ / * +ab - cdef}$$

$$\underline{ab + * cd} - / e + f$$

$$\underline{ab + cd - *} / e + f$$

$$\underline{ab + cd - * e} / + f$$

$$\underline{ab + cd - * e / f} +$$

ADT -

Enqueue(e)

Dequeue(): e

isEmpty(): -

Print_Queue(): FIFO

linear

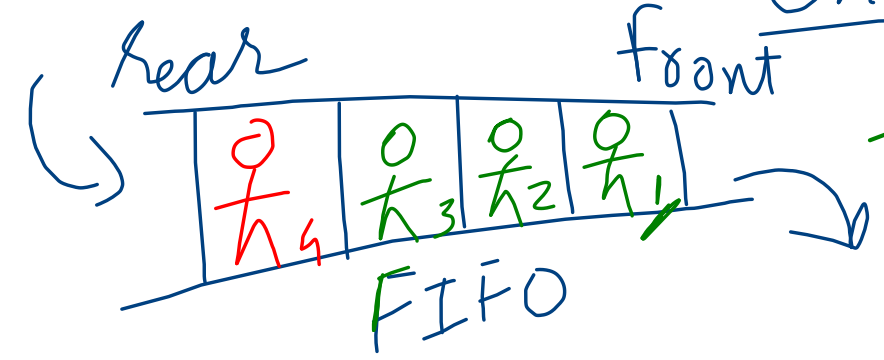
Circular

bounded buffer

Priority Queue

04
77
0000
0000
0000

Queue $\rightarrow$ linear, Double ended



$\rightarrow$ Comm
 $\rightarrow$ Streaming
 $\rightarrow$ scheduling
 $\rightarrow$ transfer

$\rightarrow$ remove-highest to lowest

for



len() - 1

for (char in "1010")

$$\begin{array}{r} 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \\ \hline 8 + 0 \qquad + 2 + 0 \end{array}$$

1 (char at a time) 10

len(b) - 1

char * 2
int() $\oplus$ sum

sum = sum +

$n = \text{len(bin)} - 1$

$n = n - 1$

$n = n - 2$

$n = n / 2$


```
def convert_to_dec(binary):
    sum=0
    n=len(binary)-1
    for bin in binary:
        sum+=(int(bin)*2**(n))
        n-=1
    return sum
```

```
binary="1110"
```

```
print(f"Decimal of {binary} is {convert_to_dec(binary)}")
```

1010

$n=3$

~~1~~ ~~1~~ ~~1~~ ~~0~~

$$\text{sum} = 0 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$$

⁰
 [10, 8, 15, 12, 20]
 Buy

⁰
 profit ($P[i] - P[j] > 0$)
 $P[j] - P[i] \checkmark$
 sell - Buy > 0

Profit = ~~0~~ 5 10 12

i
[10, 8, 15, 12,
20]

for $i = 0$ to $n-1$

for $j = i+1$ to $n-1$

$\text{profit}[P[j] - P[i] > \text{profit}]$

update profit

Day Buy
Day sell

Sort
len

→

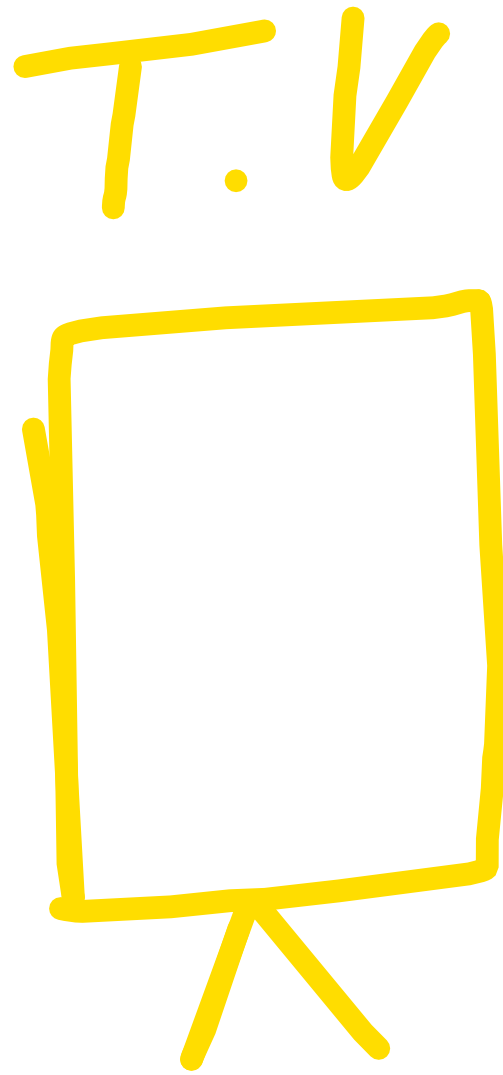
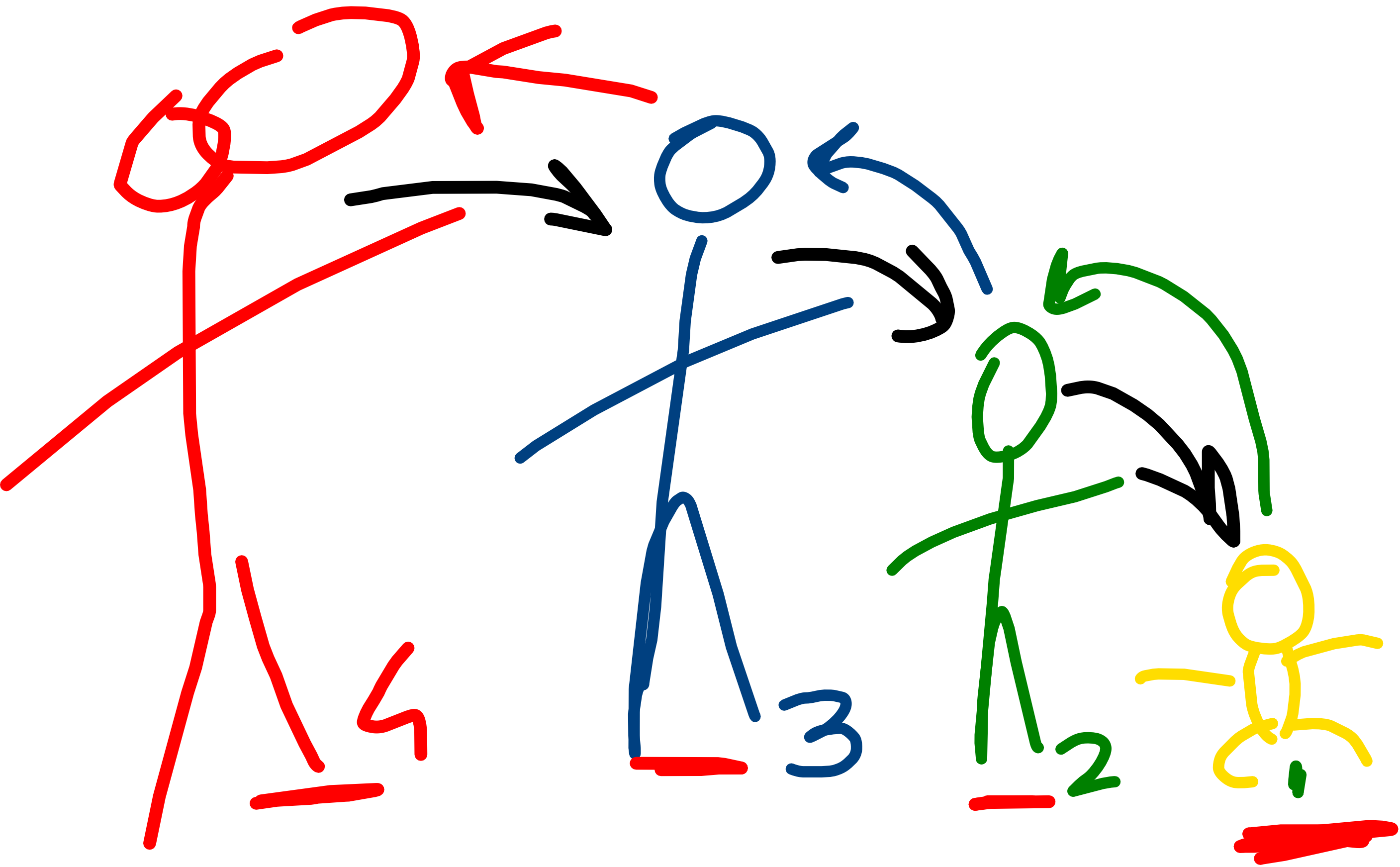
10	8	6	3	1
0	1	2	3	4

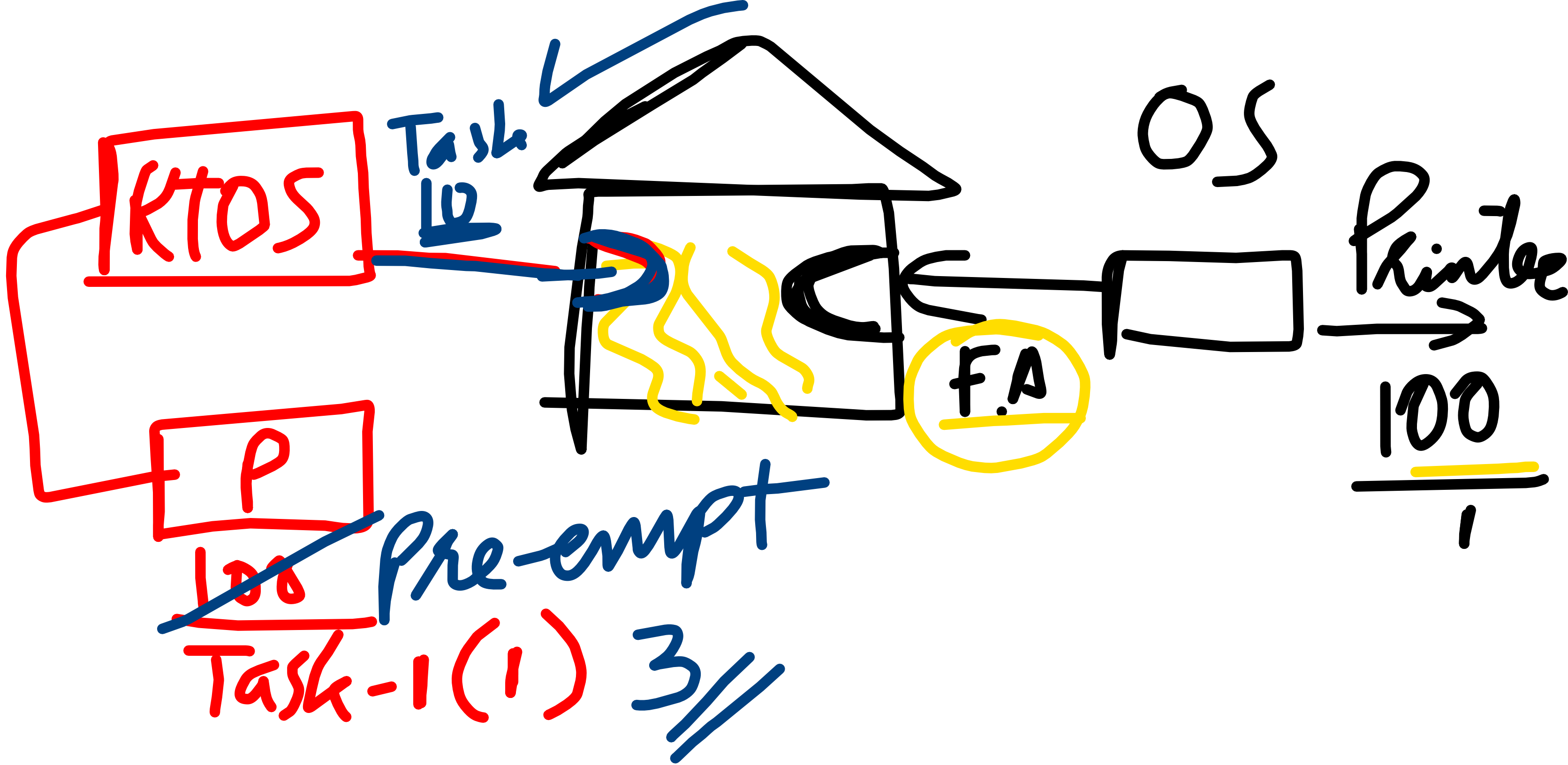
→

0	1	2	3	4
10	5	8	3	6

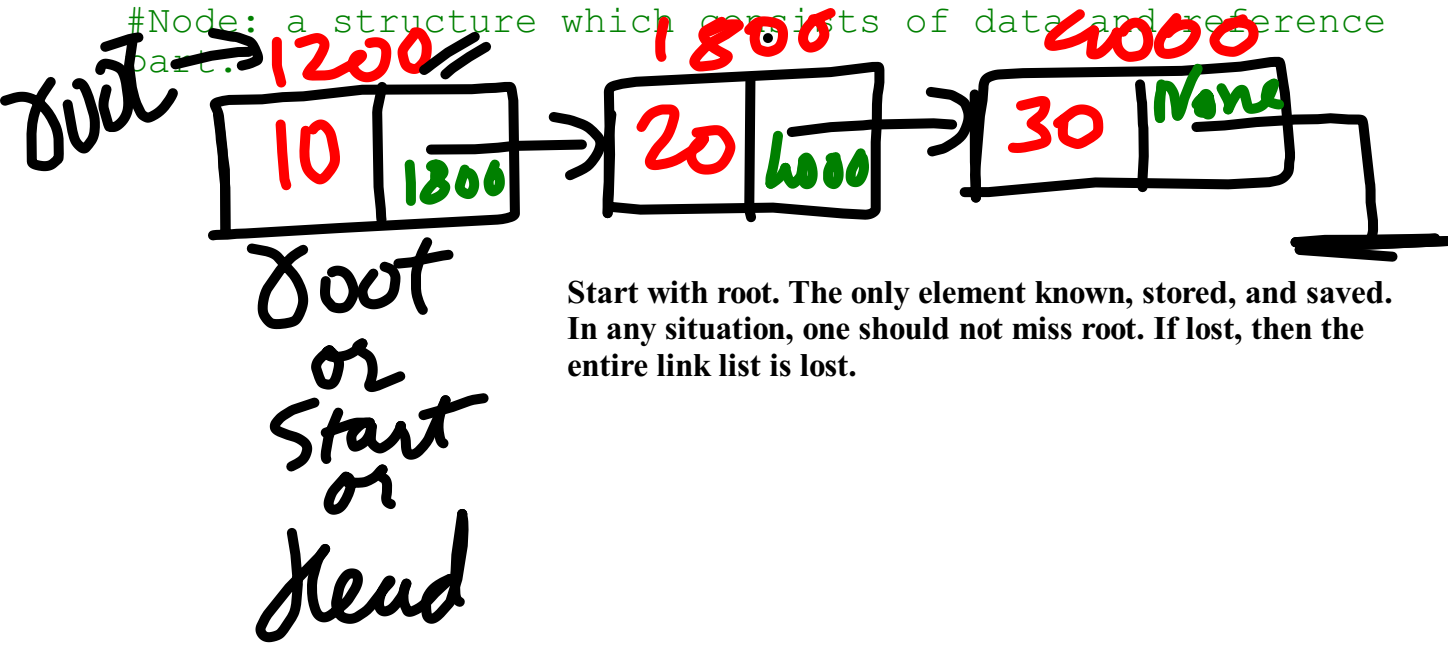
Sort

3	5	6	8	10
0				



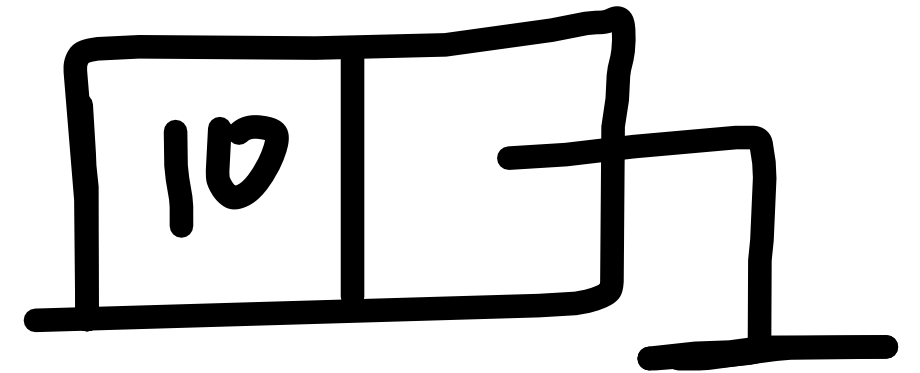


#Linked List
#Collection of nodes interconnected in a sequential manner.
#Node: a structure which consists of data and reference part.



```
class Node:
    def __init__(self,data):
        self.data=data
        self.next=None
        #self-refering pointer/reference
        #can ref to its own type
```

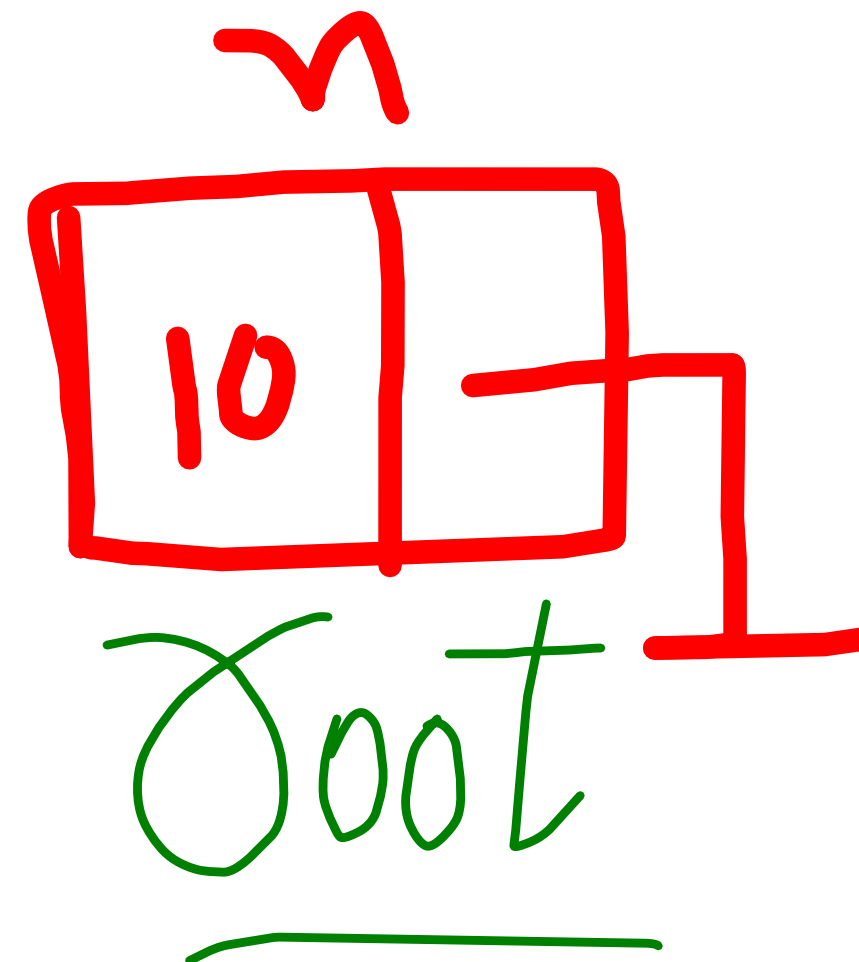
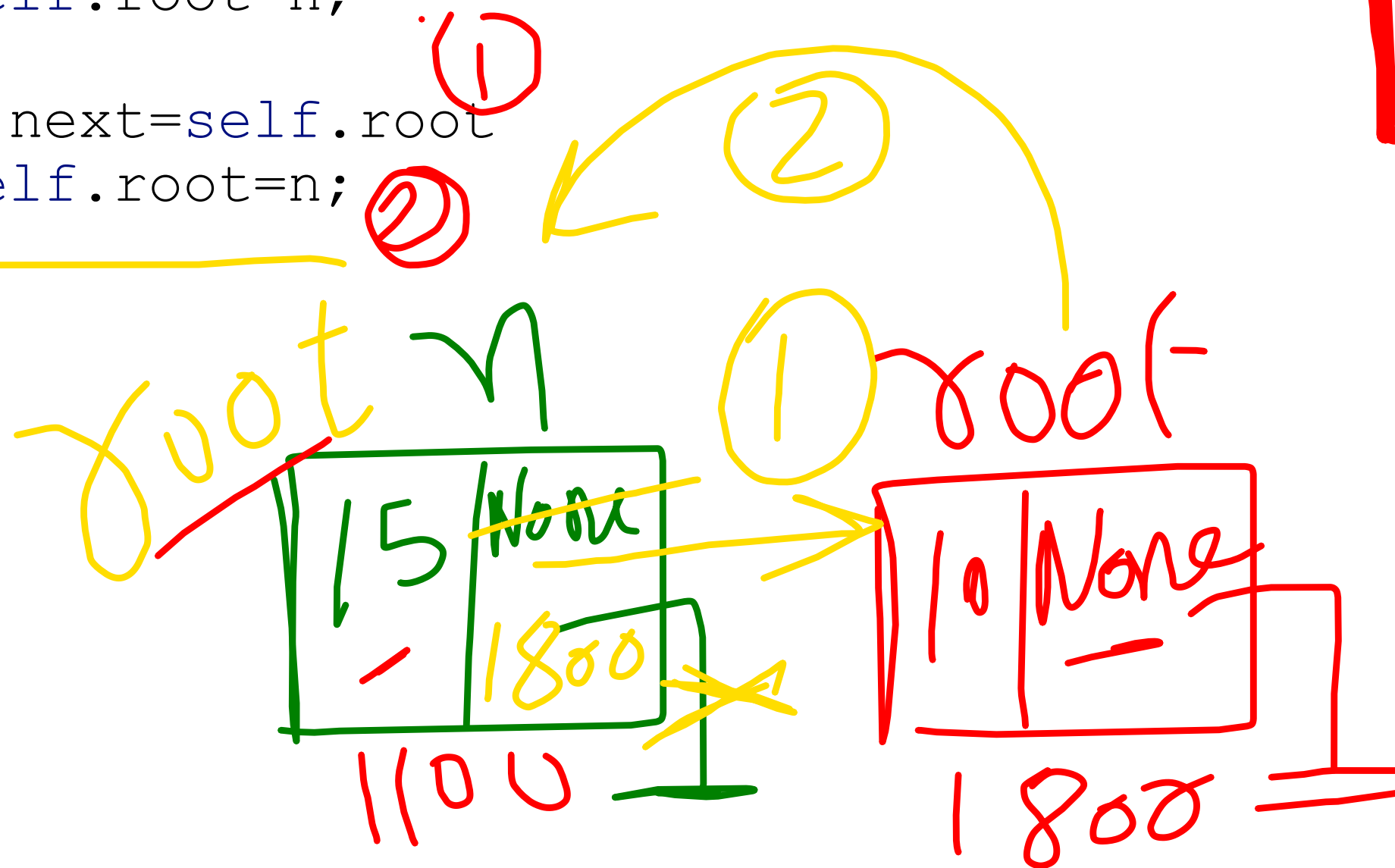
head



root

Start

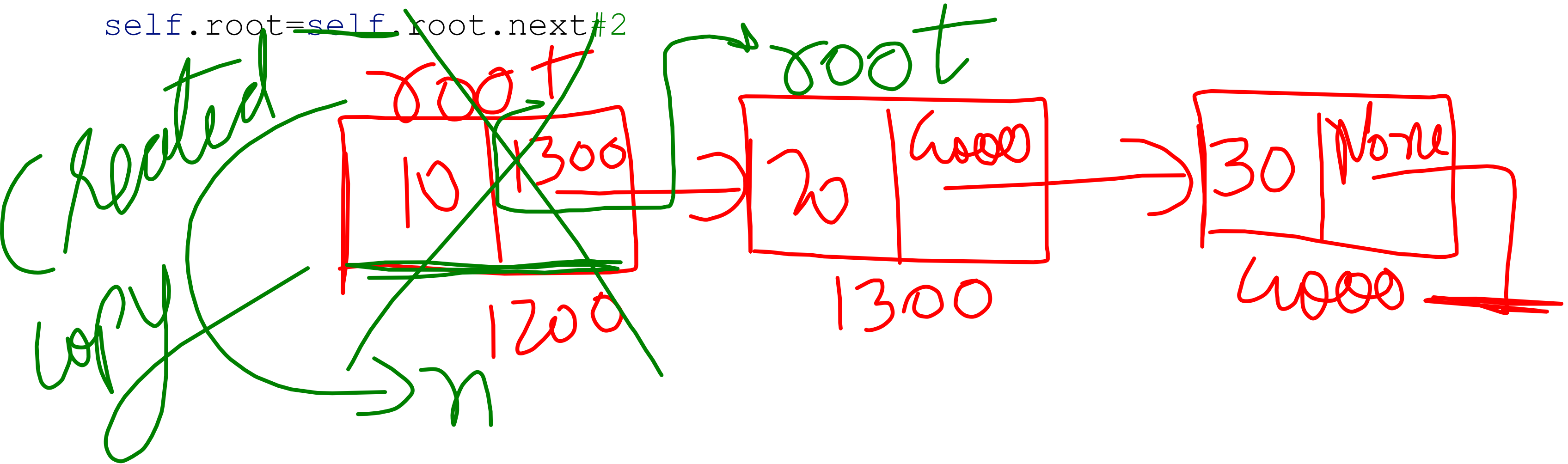

```
def insert_left(self, n):  
    if self.root == None:  
        self.root = n;  
    else:  
        ✓ n.next = self.root  
        self.root = n;
```



```
def delete_left(self):
    if self.root==None:
        print("\nEmpty list")
    else:
        n=self.root#1
```

```
self.root=self.root.next#2
```

~~Diagram of a node with root and next pointers~~
~~root=None~~




```
def insert_right(self,n):
    if self.root==None:
        self.root=n;
    else:
```

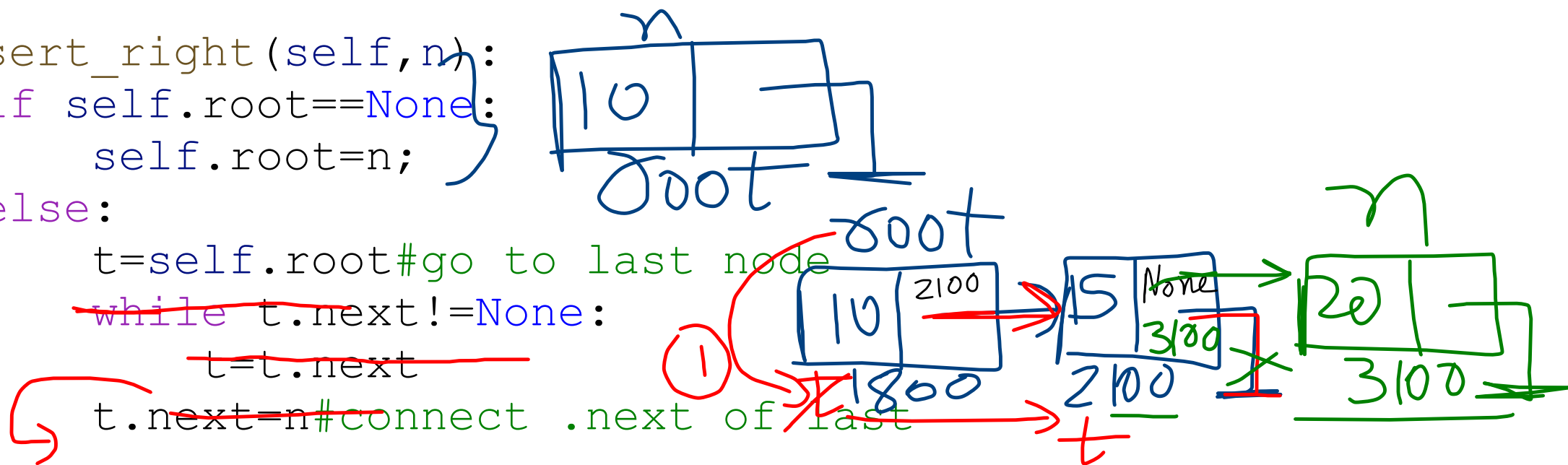
t=self.root#go to last node

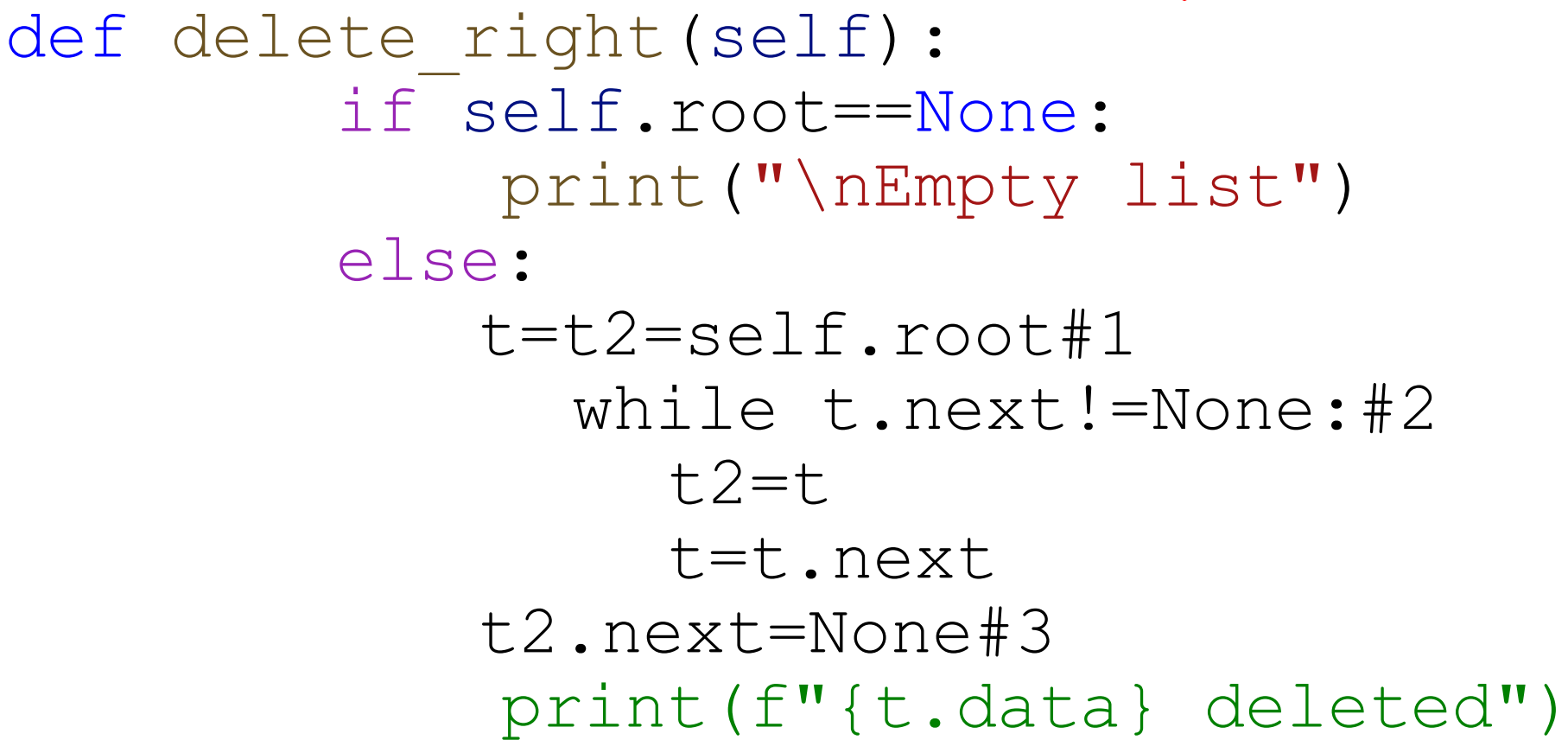
~~while t.next!=None:~~

~~t=t.next~~

~~t.next=n#connect .next of last~~

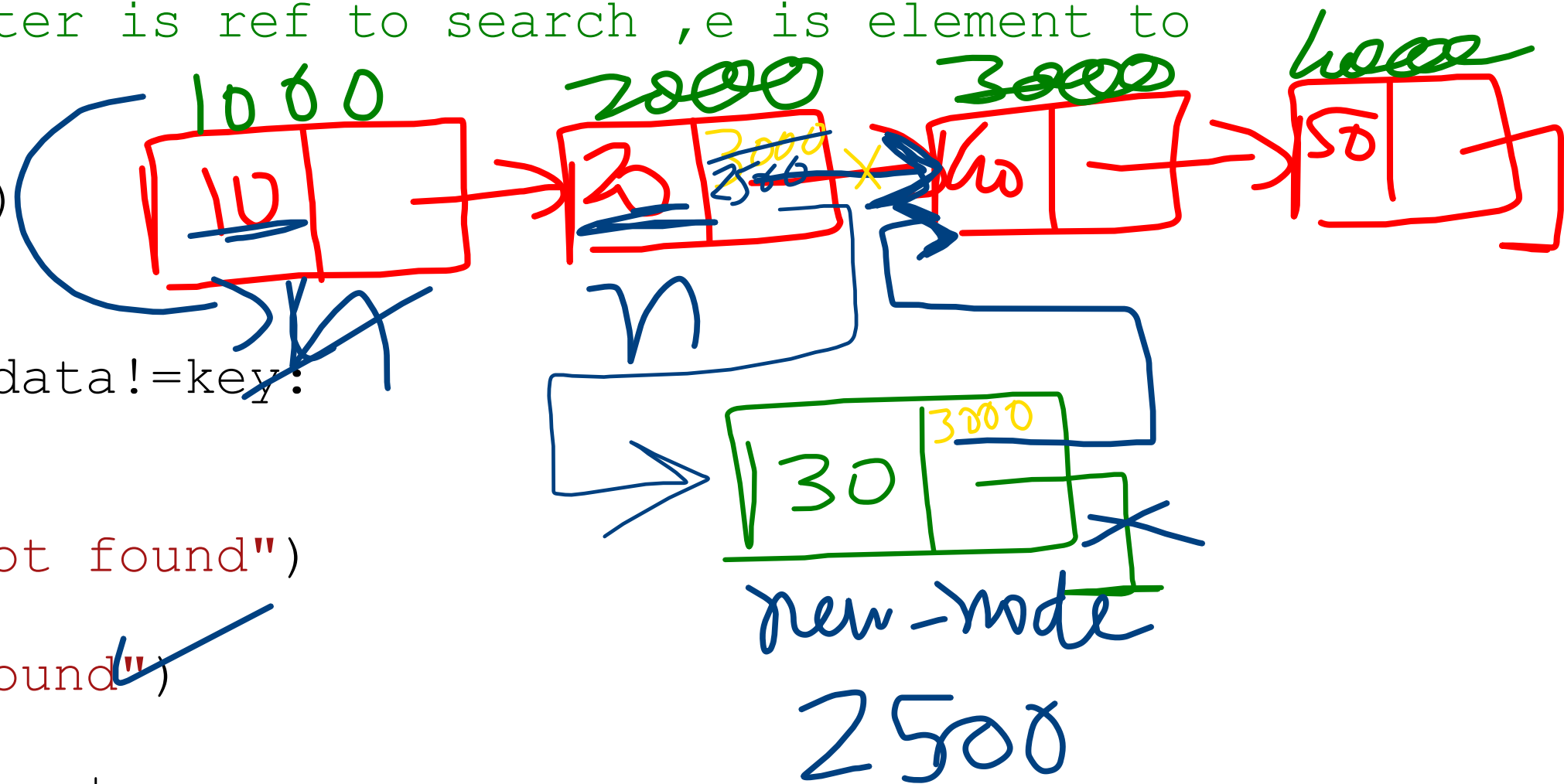
to n



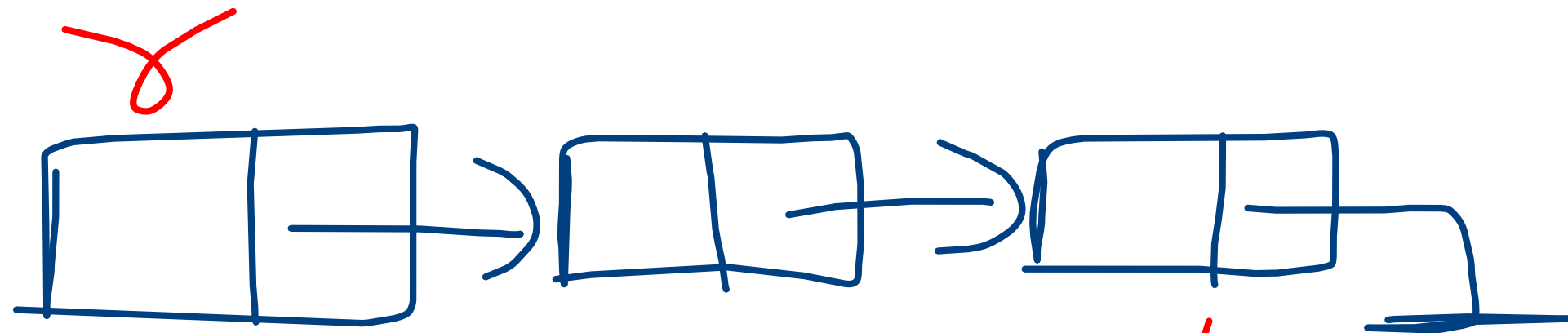


```
def insert_after(self, key, e): #after is ref to search, e is element to insert
```

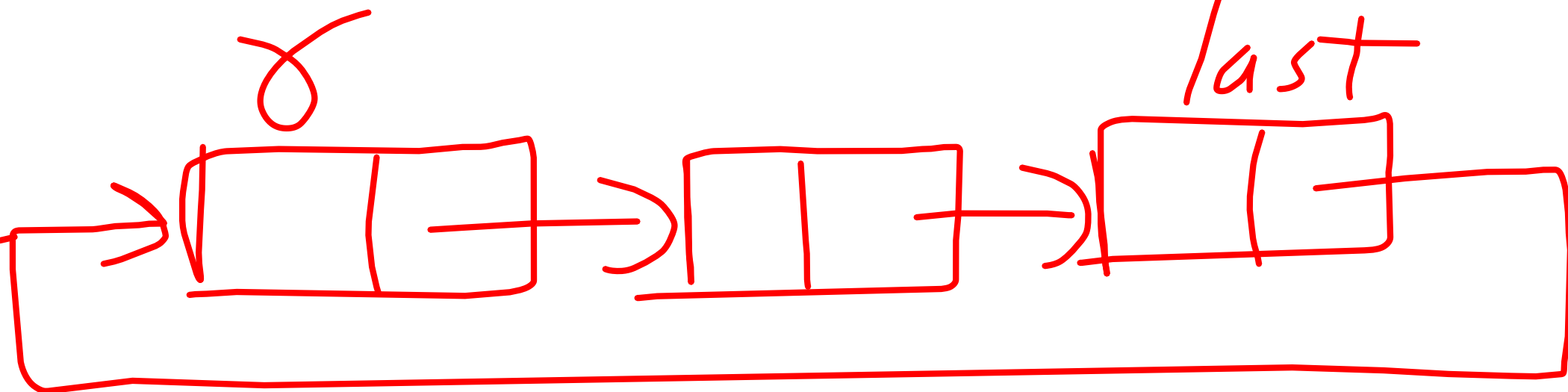
```
    if self.root == None:
        print("\nEmpty list")
    else:
        n = self.root
        while n != None and n.data != key:
            n = n.next
        if n == None:
            print("Element not found")
        else:
            print("Element found")
            new_node = Node(e)
            new_node.next = n.next
            n.next = new_node
            print("inserted....")
```



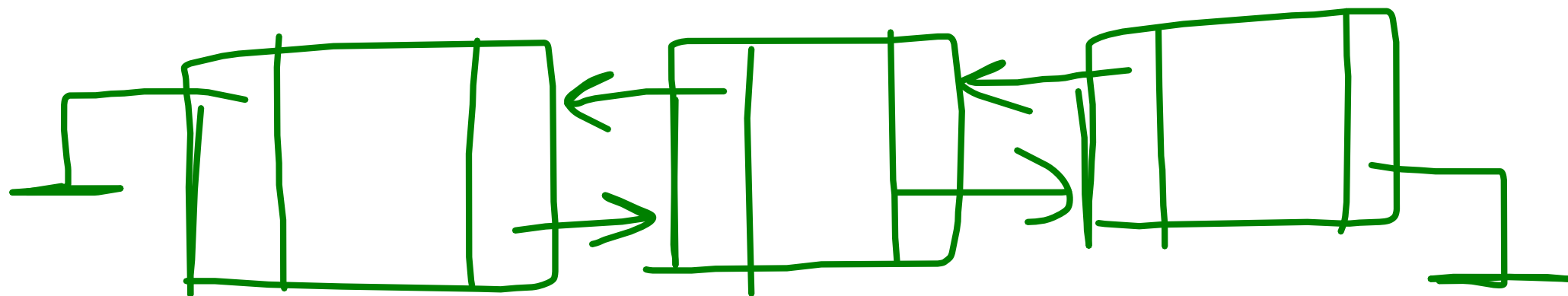
Linear



Circular



Doubly



Doubly + Circular

